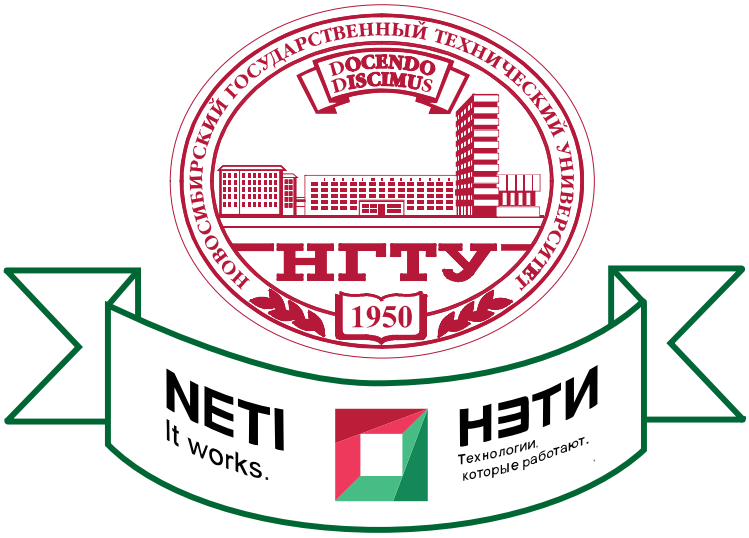
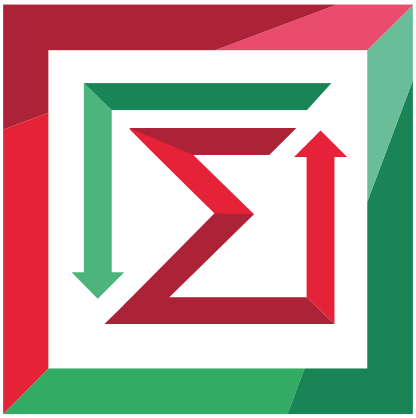
Министерство науки и высшего образования
Российской Федерации
Федеральное государственное бюджетное
образовательное учреждение высшего образования
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»



Кафедра теоретической и прикладной информатики
Лабораторная работа №3
По дисциплине «Программные средства защиты информации»

Стандарты симметричного шифрования DES и ГОСТ 28147-89



Факультет:	ПМИ
Группа:	ПМИМ-51
Студенты:	Гавриленко Андрей, Яшков Иван
Вариант:	1
Преподаватель:	Авдеенко Татьяна Владимировна

Новосибирск, 2026

1. Вариант

Реализация алгоритма AES.

2. Задание

I. Реализовать приложение для шифрования, позволяющее выполнять следующие действия:

1. Шифровать данные по заданному в варианте алгоритму:

1) шифруемый текст должен храниться в одном файле, а ключ шифрования – в другом;
2) зашифрованный текст должен сохраняться в файл;
3) в процессе шифрования предусмотреть возможность просмотра и изменения ключа, шифруемого и зашифрованного текстов в шестнадцатеричном и символьном виде;
4) программа должна показывать время шифрования.

2. Исследовать лавинный эффект (исследования проводить на одном блоке текста):

1) для бита, который будет изменяться, приложение должно позволять задавать его позицию (номер) в открытом тексте или в ключе;
2) приложение должно уметь после каждого раунда шифрования подсчитывать число бит, изменившихся в зашифрованном тексте при изменении одного бита в открытом тексте либо в ключе;
3) приложение может строить графики зависимости числа бит, изменившихся в зашифрованном тексте, от раунда шифрования, либо графики можно строить в стороннем ПО, но тогда приложение для шифрования должно сохранять в файл необходимую для построения графиков информацию.

II. Реализовать приложение для дешифрования, позволяющее выполнять следующие действия:

1. Дешифровать данные по заданному в варианте алгоритму:
1) зашифрованный текст должен храниться в одном файле, ключ – в другом;
2) расшифрованный текст должен сохраняться в файл;
3) в процессе дешифрования предусмотреть возможность просмотра и изменения ключа, зашифрованного и расшифрованного текстов в шестнадцатеричном и символьном виде.

III. Реализовать приложение, вычисляющее значения 1–4 критериев для алгоритмов DES и ГОСТ. Можно взять стороннюю реализацию того алгоритма, который не указан в варианте.

IV. С помощью реализованных приложений выполнить следующие задания:

1. Протестировать правильность работы разработанных приложений.
2. Исследовать лавинный эффект при изменении одного бита в открытом тексте и в ключе:
построить графики зависимостей числа бит, изменившихся в зашифрованном сообщении, от раунда шифрования (всего должно быть построено 2 графика).
3. Сравнить значения критериев 1–4 для алгоритмов DES и ГОСТ при изменении одного бита в блоке открытого текста и одного бита в ключе при использовании одного и того же сообщения. Сообщение должно состоять хотя бы из пяти блоков (чем больше, тем точнее будут оценки критериев 1–4).
4. Сделать выводы о проделанной работе.

2. Разработанное приложение

При выполнении лабораторной работы было написано консольное приложение на языке C, реализующее все функциональные требования, приведенные в задании под пунктами I, II, III. Некоторые функции программы приведены в конце отчета.

При запуске создаются три файла — message.txt для сообщения, key.txt для ключа, cipher.txt для шифротекста.
Основа приложения – блочный шифр AES из 10 раундов с размером блока 128 бит и размером ключа 128 бит.

Главное меню:
Choose a mode:
(1) Encrypt data from 'message.txt' with key 'key.txt' and put new message in 'cipher.txt';
(2) Decrypt data from 'cipher.txt' with key 'key.txt' and put new message in 'message.txt';
(3) Examine the results of AES (avalanche + criteria d1-d4);
(0) Exit.
>

1) При нажатии на (1), пользователь попадает в меню шифрования:
Choose a mode:
(1) Encrypt with AES-128;
(show <message.txt/cipher.txt/key.txt> <16/symbol>) Show a file in a specific form;
(write <message.txt/cipher.txt/key.txt> <16/symbol> <text>) Write text into a file in a specific form;
(0) Back.
>

Здесь пользователь может задать сообщение, ключ и даже сообщение шифра с помощью символьного, 2-ичного или 16-ичного ввода, посмотреть эти значения в символьном, 2-ичном или 16-ичном виде. При нажатии на 1 в файл cipher.txt помещается зашифрованное сообщение.

2) При нажатии на (2), выполняется дешифрование текста в cipher.txt.
Choose a mode:
(1) Decrypt with AES-128;
(0) Back.
>

При нажатии на 1 в файл message.txt помещается расшифрованное сообщение.

3) При нажатии на 3, пользователь попадает в меню исследования лавинного эффекта:
Choose a mode:
(1 <bit_pos[0..127]>) Run avalanche research for the first block (writes 2 CSVs);
(aes) Compute criteria d1-d4 for AES-128;
(gost) Compute criteria d1-d4 for GOST 28147-89;
(0) Back.
>

При нажатии <1 позиция_бита> создастся 2 CSV файла для просмотра графиков зависимости количества измененных битов в ключе и в сообщении от номера раунда.
При нажатии <aes> будут вычислены значения критериев при исследовании диффузии и конфузии для алгоритма из варианта, дополнительно будут сохранены матрицы в CSV файлы.
При нажатии <gost> будет выполнен предыдущий пункт, но для алгоритма ГОСТ 28147-89, реализация которого, в соответствии с заданием, была взята из открытых источников.

3. Задания из пункта 4:

1. Протестировать правильность работы разработанных приложений.

Для тестирования правильности работы алгоритма были взяты входные данные из документа FIPS Publication 197, в котором описывается стандарт AES. Там предлагается взять:

Сообщение: $00112233445566778899aabbccddeeff_{16}$

Ключ: $000102030405060708090a0b0c0d0e0f_{16}$

Должен получаться результат: $69c4e0d86a7b0430d8cdb78070b4c55a_{16}$

В программе в режиме шифрования были введены команды:

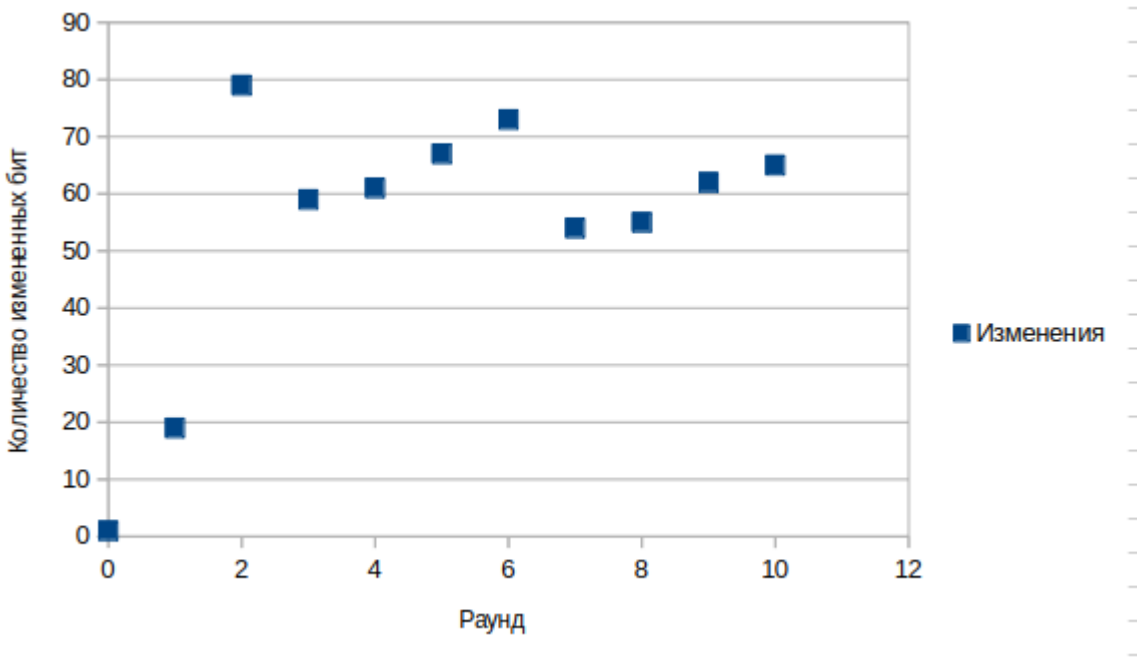
```
> write message.txt 16 00112233445566778899aabbccddeeff
> write key.txt 16 000102030405060708090a0b0c0d0e0f
> 1                       // Encrypted 16 bytes -> 16 bytes in 0.000009 s.
> show cipher.txt 16              // 69c4e0d86a7b0430d8cdb78070b4c55a
```

Дешифрование показало изначальное сообщение — AES работает.

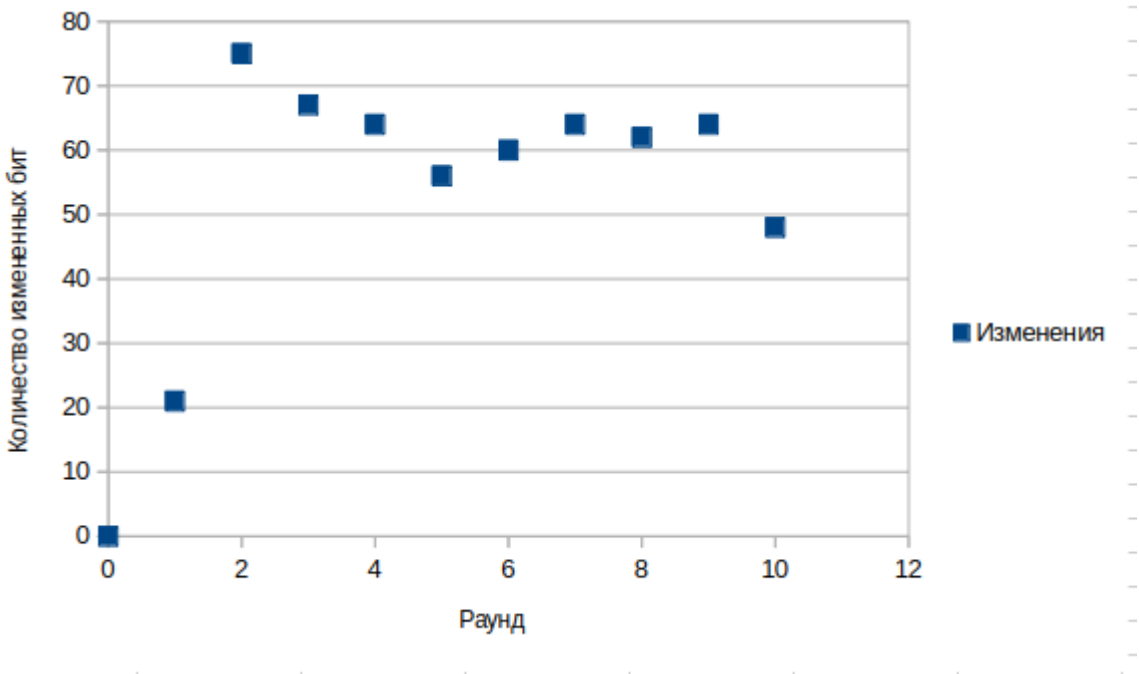
2. Исследовать лавинный эффект при изменении одного бита в открытом тексте и в ключе: построить графики зависимостей числа бит, изменившихся в зашифрованном сообщении, от раунда шифрования (всего должно быть построено 2 графика).

Для этого необходимо зайти в меню исследования и нажать <1 64> для изменения 64-го бита в блоке данных. Результаты:

Для бита, измененного в сообщении:



Для бита, измененного в ключе:



Как можно заметить, в данной работе, в отличие от предыдущей с сетью Фейстеля, изменение одного бита в сообщении меняет в среднем половину битов уже через несколько раундов.

3. Сравнить значения критериев 1–4 для алгоритмов DES и ГОСТ при изменении одного бита в блоке открытого текста и одного бита в ключе при использовании одного и того же сообщения. Сообщение должно состоять хотя бы из пяти блоков (чем больше, тем точнее будут оценки критериев 1–4).

Для тестирования было взято следующее сообщение (112 байт: 7 блоков AES, 14 блоков ГОСТ):

```
Hello, Andrey! Who is your favorite character in isaac? Cain, why? Nah i'm just asking
don't worry! Why should I
```

Для AES использовался следующий ключ (128 бит — 16 байт):

```
 be worried? Aha
```

Для ГОСТ использовался следующий ключ (256 бит — 32 байт):

```
ha I'm joking!! Don't text me...
```

Значения критериев для AES:
Diffusion (text): d1=63.8170 d2=0.9913 d3=0.9732 d4=0.6887
Confusion (key): d1=63.9408 d2=0.9922 d3=0.9729 d4=0.6873

Значения критериев для ГОСТ:
Diffusion (text): d1=31.8806 d2=1.0000 d3=0.9724 d4=0.7888
Confusion (key): d1=31.8025 d2=1.0000 d3=0.9726 d4=0.7881

Судя по критерию d1, оба шифра меняют примерно половину своего блока при изменении всего одного бита начального сообщения, что достаточно хорошо.
Критерий d2 для обоих шифров стремится к 1, что означает, что почти все биты сообщения в одном блоке меняются при изменении одного бита сообщения.

Критерий d3 стремится к 1 для обоих шифров, критерий d4 на самом деле тоже стремится к единице, просто нужно сильно больше блоков текста чтобы это показать. Из-за того, что у ГОСТ больше блоков, d4 находится ближе к единице. Оба этих критерия указывают на степень лавинного эффекта, значения близкие к единице говорят о том, что влияние одного бита в конкретном шифре достаточно велико.

4. Сделать выводы о проделанной работе.

В данной работе было разработано приложение для шифрования/дешифрования сообщения с помощью AES-128. Глядя на графики исследования лавинного эффекта, а также критерии d1, d2, d3, d4 критериев AES-128 и ГОСТ 28147-89 можно сделать следующие выводы:

1) При изменении бита открытого текста в среднем меняется половина блока, в отличие от сети Фейстеля, где образующие функции были линейны;

2) При изменении бита ключа также меняется половина блока;

3) Шифр AES обладает хорошей степенью лавинного эффекта, что было показано при исследовании диффузии и конфузии, что обеспечивает криптостойкость шифра, делая шифротекст невозможным для предсказания.

4) Шифр ГОСТ 28147-89 также показывает хорошие значения, соответствующие его характеристикам. Оба шифра являются криптостойкими благодаря высокой степени лавинного эффекта.

4. Несколько основных функций

1) Функция шифрования одного блока AES:

```
void aes_encrypt_block(const uint8_t in[AES_BLOCK_BYTES],
                       uint8_t out[AES_BLOCK_BYTES],
                       const uint8_t key[AES_KEY_BYTES]) {
  uint8_t rk[AES_ROUNDS + 1][AES_BLOCK_BYTES];
  key_expansion(key, rk);

  uint8_t state[16];
  memcpy(state, in, 16);

  add_round_key(state, rk[0]);
  for (int r = 1; r <= AES_ROUNDS - 1; r++) {
    sub_bytes(state);
    shift_rows(state);
    mix_columns(state);
    add_round_key(state, rk[r]);
  }
  sub_bytes(state);
  shift_rows(state);
  add_round_key(state, rk[AES_ROUNDS]);

  memcpy(out, state, 16);
}
```

2) Функция разворачивания ключа AES:

```
static void key_expansion(const uint8_t key[AES_KEY_BYTES],
                          uint8_t rk[AES_ROUNDS + 1][AES_BLOCK_BYTES]) {
  memcpy(rk[0], key, AES_KEY_BYTES);

  for (int r = 1; r <= AES_ROUNDS; r++) {
    uint8_t t[4];
    t[0] = rk[r - 1][12];
    t[1] = rk[r - 1][13];
    t[2] = rk[r - 1][14];
    t[3] = rk[r - 1][15];

    uint8_t tmp = t[0];
    t[0] = t[1];
```

```
    t[1] = t[2];
    t[2] = t[3];
    t[3] = tmp;

    t[0] = sbox[t[0]];
    t[1] = sbox[t[1]];
    t[2] = sbox[t[2]];
    t[3] = sbox[t[3]];

    t[0] ^= rcon[r];

    for (int i = 0; i < 4; i++) rk[r][i] = (uint8_t)(rk[r - 1][i] ^ t[i]);
    for (int i = 4; i < 16; i++)
      rk[r][i] = (uint8_t)(rk[r - 1][i] ^ rk[r][i - 4]);
  }
}
```

3) Функция для перемешивания столбцов AES:

```
static void mix_columns(uint8_t state[16]) {
  for (int c = 0; c < 4; c++) {
    uint8_t* col = state + c * 4;
    uint8_t a0 = col[0], a1 = col[1], a2 = col[2], a3 = col[3];
    col[0] = xtime(a0) ^ (xtime(a1) ^ a1) ^ a2 ^ a3;
    col[1] = a0 ^ xtime(a1) ^ (xtime(a2) ^ a2) ^ a3;
    col[2] = a0 ^ a1 ^ xtime(a2) ^ (xtime(a3) ^ a3);
    col[3] = (xtime(a0) ^ a0) ^ a1 ^ a2 ^ xtime(a3);
  }
}
```

4) Функция исследования лавинного эффекта (для графиков):

```
static int run_avalanche(int bit_pos) {
  unsigned char message[BUFFER_SIZE] = {0};
  unsigned char key[FEISTEL_KEY_BYTES] = {0};

  int fd_m = open(MESSAGE_FILE, O_RDONLY);
  int fd_k = open(KEY_FILE, O_RDONLY);
  if (fd_m == -1 || fd_k == -1) {
    perror("open");
    if (fd_m != -1) close(fd_m);
    if (fd_k != -1) close(fd_k);
    return FAILURE;
  }
  read(fd_m, message, BUFFER_SIZE);
  read(fd_k, key, FEISTEL_KEY_BYTES);
  close(fd_m);
  close(fd_k);

  printf("Avalanche research for bit position %d:\n", bit_pos);

  for (int km = 1; km <= 2; km++) {
    for (int fn = 1; fn <= 2; fn++) {
      feistel_settings s = {km, fn};

      char fname[96];
      snprintf(fname, sizeof(fname), "avalanche_km%d_fn%d_bit%d_ptext.csv", km,
               fn, bit_pos);
      run_case(message, key, 0, bit_pos, &s, fname);

      snprintf(fname, sizeof(fname), "avalanche_km%d_fn%d_bit%d_key.csv", km,
               fn, bit_pos);
      run_case(message, key, 1, bit_pos, &s, fname);
    }
  }

  printf("8 CSV files written for bit %d.\n", bit_pos);
  return SUCCESS;
}
```

5) Функция для вычисления критериев:

```
static int run_criteria(void) {
  const int n_bits = 128;
  const int m_bits = 128;

  uint8_t* data = NULL;
  uint8_t* padded = NULL;
  uint8_t* Y_all = NULL;
  uint32_t** a_text = NULL;
  uint32_t** b_text = NULL;
  uint32_t** a_key = NULL;
  uint32_t** b_key = NULL;

  data = malloc(BUFFER_SIZE);
  if (!data) {
    perror("malloc");
    return FAILURE;
  }

  int fd = open(MESSAGE_FILE, O_RDONLY);
  if (fd == -1) {
    perror("open message");
    free(data);
    return FAILURE;
  }
  ssize_t n = read(fd, data, BUFFER_SIZE);
  close(fd);
  if (n < 0) n = 0;
  size_t data_len = (size_t)n;

  size_t N = (data_len + AES_BLOCK_BYTES - 1) / AES_BLOCK_BYTES;
  if (N == 0) N = 1;
  size_t padded_len = N * AES_BLOCK_BYTES;

  padded = calloc(padded_len, 1);
  if (!padded) {
    perror("calloc");
    free(data);
    return FAILURE;
  }
  memcpy(padded, data, data_len);
  free(data);
  data = NULL;

  uint8_t key[AES_KEY_BYTES];
  if (load_key(key) == FAILURE) {
    free(padded);
    return FAILURE;
  }

  Y_all = malloc(N * AES_BLOCK_BYTES);
  if (!Y_all) {
    perror("malloc");
    free(padded);
    return FAILURE;
  }

  for (size_t b = 0; b < N; b++)
    aes_encrypt_block(padded + b * AES_BLOCK_BYTES, Y_all + b * AES_BLOCK_BYTES,
                      key);

  a_text = calloc((size_t)n_bits, sizeof(uint32_t*));
  b_text = calloc((size_t)n_bits, sizeof(uint32_t*));
  a_key = calloc((size_t)n_bits, sizeof(uint32_t*));
  b_key = calloc((size_t)n_bits, sizeof(uint32_t*));
```

```
if (!a_text || !b_text || !a_key || !b_key) {
  perror("calloc");
  free(padded);
  free(Y_all);
  free_matrices(a_text, b_text, a_key, b_key, n_bits);
  return FAILURE;
}

int alloc_ok = 1;
for (int i = 0; i < n_bits; i++) {
  a_text[i] = calloc((size_t)m_bits, sizeof(uint32_t));
  b_text[i] = calloc((size_t)m_bits + 1, sizeof(uint32_t));
  a_key[i] = calloc((size_t)m_bits, sizeof(uint32_t));
  b_key[i] = calloc((size_t)m_bits + 1, sizeof(uint32_t));
  if (!a_text[i] || !b_text[i] || !a_key[i] || !b_key[i]) {
    alloc_ok = 0;
    break;
  }
}
if (!alloc_ok) {
  perror("calloc");
  free(padded);
  free(Y_all);
  free_matrices(a_text, b_text, a_key, b_key, n_bits);
  return FAILURE;
}

for (int i = 0; i < n_bits; i++) {
  for (size_t b = 0; b < N; b++) {
    uint8_t X[AES_BLOCK_BYTES];
    memcpy(X, padded + b * AES_BLOCK_BYTES, AES_BLOCK_BYTES);
    flip_bit(X, i);

    uint8_t Y[AES_BLOCK_BYTES];
    aes_encrypt_block(X, Y, key);

    uint8_t* Yb = Y_all + b * AES_BLOCK_BYTES;
    int w = count_diff_bits(Y, Yb, AES_BLOCK_BYTES);
    b_text[i][w]++;

    for (int j = 0; j < m_bits; j++) {
      uint8_t x = (uint8_t)(Y[j / 8] ^ Yb[j / 8]);
      if (x & (1 << (7 - (j % 8)))) a_text[i][j]++;
    }
  }
}

for (int i = 0; i < n_bits; i++) {
  uint8_t key_mod[AES_KEY_BYTES];
  memcpy(key_mod, key, AES_KEY_BYTES);
  flip_bit(key_mod, i);

  for (size_t b = 0; b < N; b++) {
    uint8_t Y[AES_BLOCK_BYTES];
    aes_encrypt_block(padded + b * AES_BLOCK_BYTES, Y, key_mod);

    uint8_t* Yb = Y_all + b * AES_BLOCK_BYTES;
    int w = count_diff_bits(Y, Yb, AES_BLOCK_BYTES);
    b_key[i][w]++;

    for (int j = 0; j < m_bits; j++) {
      uint8_t x = (uint8_t)(Y[j / 8] ^ Yb[j / 8]);
      if (x & (1 << (7 - (j % 8)))) a_key[i][j]++;
    }
  }
}

double s1_text = 0, s1_key = 0;
```

```
  for (int i = 0; i < n_bits; i++) {
    double sj_t = 0, sj_k = 0;
    for (int j = 0; j <= m_bits; j++) {
      sj_t += (double)j * b_text[i][j];
      sj_k += (double)j * b_key[i][j];
    }
    s1_text += sj_t / (double)N;
    s1_key += sj_k / (double)N;
  }
  double d1_text = s1_text / n_bits;
  double d1_key = s1_key / n_bits;

  long zero_a_text = 0, zero_a_key = 0;
  for (int i = 0; i < n_bits; i++) {
    for (int j = 0; j < m_bits; j++) {
      if (a_text[i][j] == 0) zero_a_text++;
      if (a_key[i][j] == 0) zero_a_key++;
    }
  }
  double d2_text = 1.0 - (double)zero_a_text / ((double)n_bits * m_bits);
  double d2_key = 1.0 - (double)zero_a_key / ((double)n_bits * m_bits);

  double s3_text = 0, s3_key = 0;
  for (int i = 0; i < n_bits; i++) {
    double inner_t = 0, inner_k = 0;
    for (int j = 0; j <= m_bits; j++) {
      inner_t += 2.0 * j * b_text[i][j];
      inner_k += 2.0 * j * b_key[i][j];
    }
    s3_text += fabs(inner_t / (double)N - m_bits);
    s3_key += fabs(inner_k / (double)N - m_bits);
  }
  double d3_text = 1.0 - s3_text / ((double)n_bits * m_bits);
  double d3_key = 1.0 - s3_key / ((double)n_bits * m_bits);

  double s4_text = 0, s4_key = 0;
  for (int i = 0; i < n_bits; i++) {
    for (int j = 0; j < m_bits; j++) {
      s4_text += fabs(2.0 * a_text[i][j] / (double)N - 1.0);
      s4_key += fabs(2.0 * a_key[i][j] / (double)N - 1.0);
    }
  }
  double d4_text = 1.0 - s4_text / ((double)n_bits * m_bits);
  double d4_key = 1.0 - s4_key / ((double)n_bits * m_bits);

  printf("\n=== Criteria (N = %zu blocks) ===\n", N);
  printf("Diffusion (text):  d1=%.4f  d2=%.4f  d3=%.4f  d4=%.4f\n", d1_text,
         d2_text, d3_text, d4_text);
  printf("Confusion (key):   d1=%.4f  d2=%.4f  d3=%.4f  d4=%.4f\n", d1_key,
         d2_key, d3_key, d4_key);

  save_matrix("criteria_a_text.csv", a_text, n_bits, m_bits);
  save_matrix("criteria_b_text.csv", b_text, n_bits, m_bits + 1);
  save_matrix("criteria_a_key.csv", a_key, n_bits, m_bits);
  save_matrix("criteria_b_key.csv", b_key, n_bits, m_bits + 1);
  printf(
      "Matrices written to criteria_a_text.csv, criteria_b_text.csv, "
      "criteria_a_key.csv, criteria_b_key.csv\n");

  free_matrices(a_text, b_text, a_key, b_key, n_bits);
  free(padded);
  free(Y_all);

  return SUCCESS;
}
```